

LECTURE – CONDITIONAL STATEMENTS

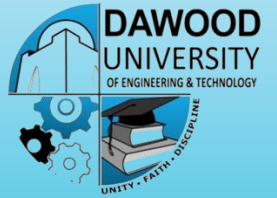
OOP (JAVA)



MR. PREM KUMAR
LECTURER
DEPARTMENT OF CYBER SECURITY

Dawood University of Engineering and Technology, Karachi

AGENDA

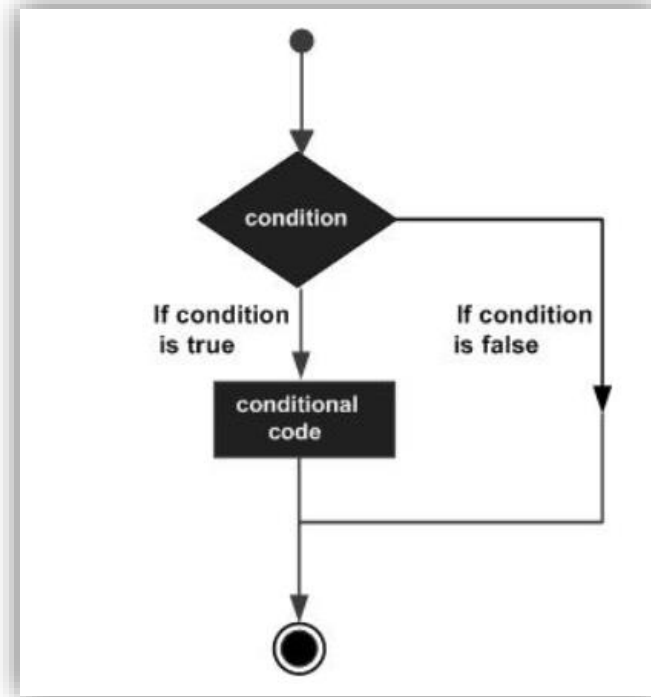


- Operators
- Types of Operator
- Keywords

Conditional Statements

■ Java supports the usual logical conditions from mathematics:

- **Less than:** $a < b$
- **Less than or equal to:** $a \leq b$
- **Greater than:** $a > b$
- **Greater than or equal to:** $a \geq b$
- **Equal to** $a == b$
- **Not Equal to:** $a \neq b$



- can use these conditions to perform different actions for different decisions.

The if Statement

- Use the if statement to specify a block of Java code to be executed if a condition is true.

- **Syntax**

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

- **Example**

```
if (20 > 18) {  
    System.out.println("20 is greater than 18");  
}
```

The if Statement

- Use the if statement to specify a block of Java code to be executed if a condition is true.

```
int x = 20;  
int y = 18;  
if (x > y) {  
    System.out.println("x is greater than y");  
}
```

Example:

Result:

```
public class Main {  
    public static void main(String[] args) {  
        int x = 20;  
        int y = 18;  
        if (x > y) {  
            System.out.println("x is greater than y");  
        }  
    }  
}
```

x is greater than y

The else Statement

- Use the else statement to specify a block of Java code to be executed if a condition is false.
- Syntax

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

The else Statement

- Use the else statement to specify a block of Java code to be executed if a condition is false.

```
int time = 20;  
if (time < 18) {  
    System.out.println("Good day.");  
} else {  
    System.out.println("Good evening.");  
}
```

Example:

```
public class Main {  
    public static void main(String[] args) {  
        int time = 20;  
        if (time < 18) {  
            System.out.println("Good day.");  
        } else {  
            System.out.println("Good evening.");  
        }  
    }  
}
```

Result:

Good evening.

The else if Statement

- Use the else if statement to specify a new condition if the first condition is false.
- Syntax

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```


The else if Statement

- Use the else if statement to specify a new condition if the first condition is false.

```
int time = 22;  
if (time < 10) {  
    System.out.println("Good morning.");  
} else if (time < 20) {  
    System.out.println("Good day.");  
} else {  
    System.out.println("Good evening.");  
}
```

Example:

```
public class Main {  
    public static void main(String[] args) {  
        int time = 22;  
        if (time < 10) {  
            System.out.println("Good morning.");  
        } else if (time < 20) {  
            System.out.println("Good day.");  
        } else {  

```

Result:

Good evening.

The short hand if else Statement (Ternary Operator)

- It consists of three operands.
- It can be used to replace multiple lines of code with a single line.
- It is often used to replace simple if else statements

Example

```
variable = (condition) ? expressionTrue : expressionFalse;
```

The short hand if else Statement (Ternary Operator)

- It can be used to replace multiple lines of code with a single line.

```
int time = 20;  
if (time < 18) {  
    System.out.println("Good day.");  
} else {  
    System.out.println("Good evening.");  
}
```

Good evening.

Example:

```
int time = 20;  
String result = (time < 18) ? "Good day." : "Good evening.";  
System.out.println(result);
```

Result:

Good evening.

```
PREM KUMAR public class Main {  
    public static void main(String[] args) {  
        int time = 20;  
        String result;  
        result = (time < 18) ? "Good day." : "Good evening.";  
        System.out.println(result);  
    }  
}
```

The Switch Statement

- Use the switch statement to select one of many code blocks to be executed.
- Syntax

```
switch(expression) {  
    case x:  
        // code block  
        break;  
    case y:  
        // code block  
        break;  
    default:  
        // code block  
}
```

The switch expression is evaluated once.

The value of the expression is compared with the values of each case.

If there is a match, the associated block of code is executed.

The break and default keywords are optional, and will be described later.

The Switch Statement



Example:

```
int day = 4;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    case 4:
        System.out.println("Thursday");
        break;
    case 5:
        System.out.println("Friday");
        break;
    case 6:
        System.out.println("Saturday");
        break;
    case 7:
```

PREM KUMAR

Result:

Thursday

```
public class Main {
    public static void main(String[] args) {
        int day = 4;
        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            case 4:
                System.out.println("Thursday");
                break;
            case 5:
                System.out.println("Friday");
                break;
            case 6:
                System.out.println("Saturday");
                break;
            case 7:
                System.out.println("Sunday");
                break;
        }
    }
}
```

The Switch Statement

The break Keyword

- When it reaches a break keyword, it breaks out of the switch block.
- This will stop the execution of more code and case testing inside the block.
- When a match is found, and the job is done, it's time for a break. There is no need for more testing.

- Syntax

```
switch(expression) {  
    case x:  
        // code block  
        break;  
    case y:  
        // code block  
        break;  
    default:  
        // code block  
}
```

The Switch Statement

The default Keyword

- The default keyword specifies some code to run if there is no case match:.

Example:

```
int day = 4;
switch (day) {
    case 6:
        System.out.println("Today is Saturday");
        break;
    case 7:
        System.out.println("Today is Sunday");
        break;
    default:
        System.out.println("Looking forward to the Weekend");
}
```

PREM KUMAR

Result:

Looking forward to the Weekend

```
public class Main {
    public static void main(String[] args) {
        int day = 4;
        switch (day) {
            case 6:
                System.out.println("Today is Saturday");
                break;
            case 7:
                System.out.println("Today is Sunday");
                break;
            default:
                System.out.println("Looking forward to the Weekend");
        }
    }
}
```

Thanks 😊